

15주 차 보고서

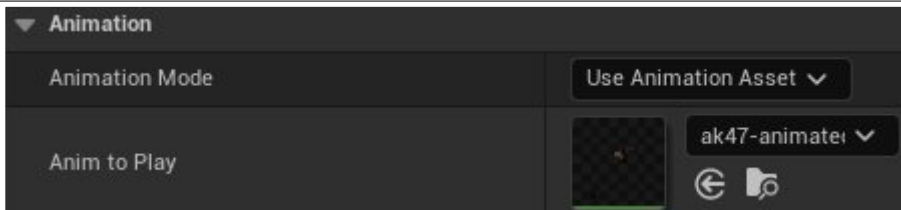
(2026년 4 월 5 일 ~ 2026년 4 월 11 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일



총 애니메이션을 적용하려고 보니까 테스트용으로 애니메이션이 한번만 적용이 되도록 해놓고 그 총을 맵에다가 두는 방식을 사용했다. 신우와 전화를 했더니 총에 애니메이션이 적용되어도 에셋을 잘못사서 밑에 이상한게 보인다고 해서 다른 총으로 바꾸기로 했다.

```

bool Room::HandleEnterPlayer(PlayerRef player)
{
    bool success = EnterRoom(player, true);

    if (success == false)
        return false;

    Protocol::S_SPAWN_ITEM spawnItemPkt;
    Protocol::ObjectInfo* itemInfo = spawnItemPkt.add_items();

    itemInfo->set_object_id(1); // 이 무기의 고유 번호는 1번!
    itemInfo->set_object_type(Protocol::OBJECT_TYPE_ITEM);
    itemInfo->set_weapon_type(Protocol::WEAPON_TYPE_RIFLE);

    Protocol::PosInfo* pos = itemInfo->mutable_pos_info();
    pos->set_x(-860.0f);
    pos->set_y(-180.0f);
    pos->set_z(30.0f);
    pos->set_yaw(0.0f);

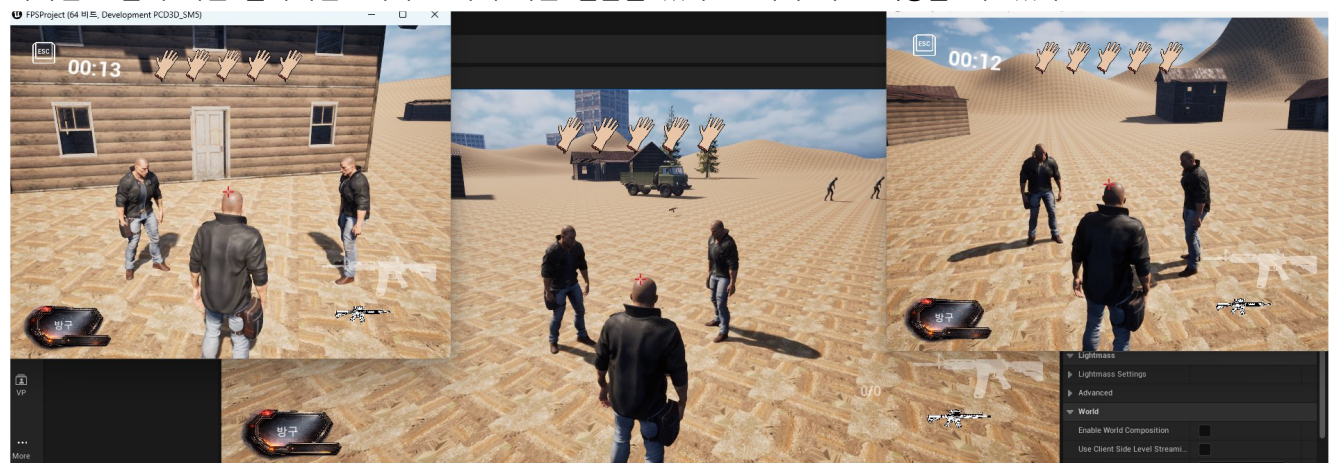
    // 방금 접속한 플레이어에게 패킷 전송
    SendBufferRef itemBuffer = ServerPacketHandler::MakeSendBuffer(spawnItemPkt);
    player->session.lock()->Send(itemBuffer);

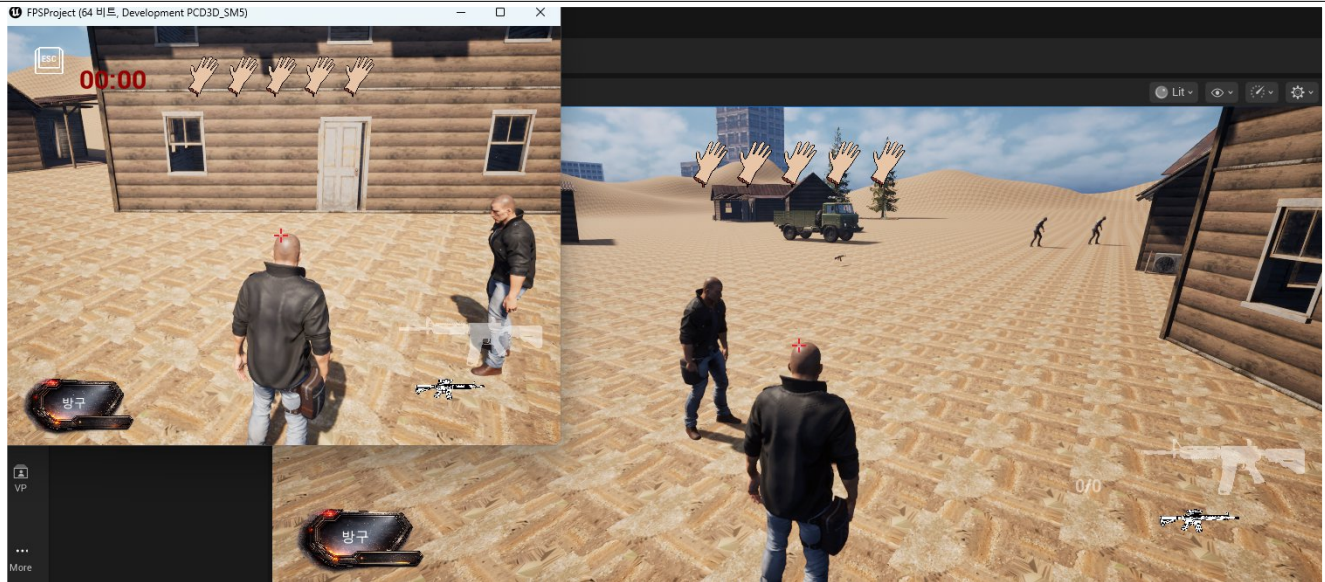
    return EnterRoom(player, true);
}

```

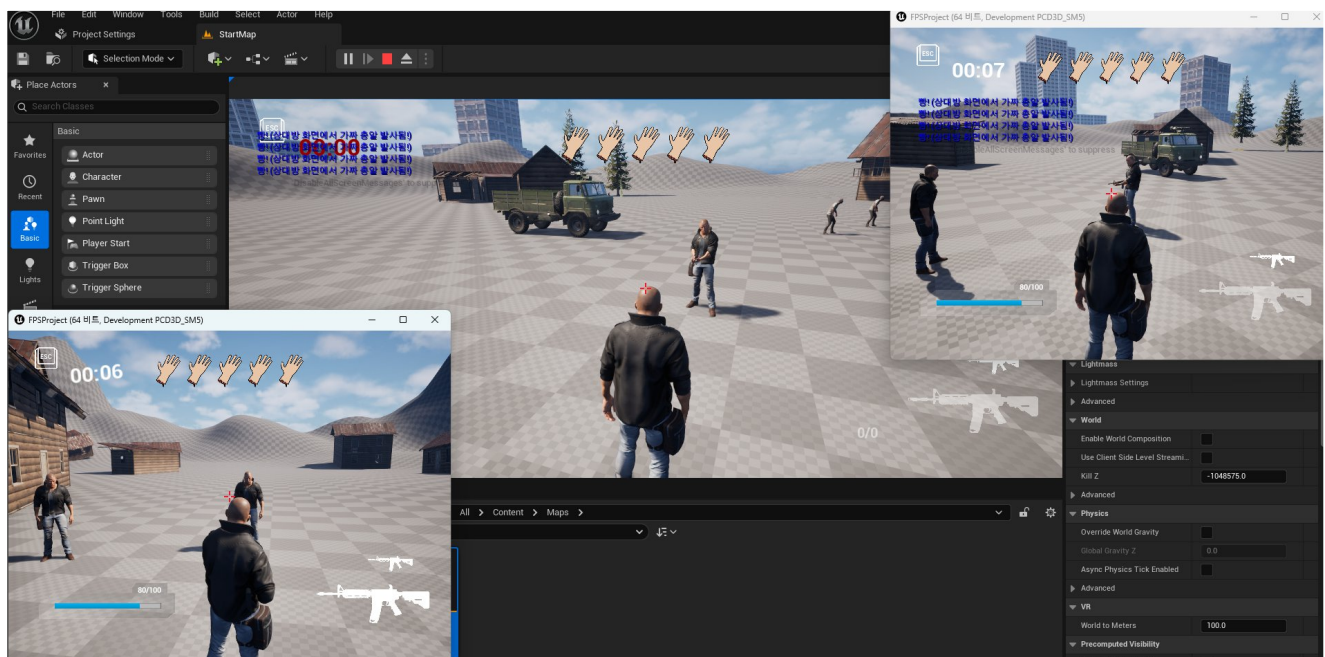
지금은 총을 맵에다가 두는 방식이 아니라 플레이어가 방에 들어가면 서버에서 지정한 좌표로 뿌려주고 그 위치에 아이템이 생성되는 방식으로 수정을 했다. 현재 맵에는 아이템이 총 하나인 상태라 이렇게 코드를 짰는데 아이템이 여러 개가 되면 서버에서 아이템을 관리하는 시스템을 만들어야 할 것 같다.

게임서버프로그래밍 시간에 체스게임을 만드는데 클라이언트가 접속을 끊었을 때 해당 플레이어가 사라지게 하고, 사라진 모습이 다른 클라이언트에서 보이게 하는 실습을 했다. 그래서 바로 적용을 해보았다.





클라 하나를 종료하면 다른 화면에서도 적용이 되는 모습이다.



총 에셋을 바꾸고 총을 주웠을 때 보이지 않는 문제가 있었는데 전에 있던 총의 크기를 0.15배 해놓아서 안보였던 거였다. 다시 1배로 하니깐 보였다. 총 발사한것도 브로드캐스트하여서 총을 쏜 캐릭터 화면 외에 나머지 화면에 땡! 하는 로그가 보이는 모습이다.

이제 총 동기화는 끝났는데 전체적인 흐름은 이렇다.

1. 서버에서 플레이어가 방에 접속하면 맵에 배치될 아이템 정보를 담은 패킷 전송, 클라는 지정된 좌표에 무기 스폰 및 관리를 위해 FieldItems맵에 아이템 ID와 액터 포인터 등록
2. 클라에서 플레이어가 무기를 주우면 서버로 패킷 전송, 서버는 해당 플레이어의 무기 정보를 갱신하고, 방 안의 모든 유저에게 브로드캐스트
3. 패킷을 받은 클라는 기존 아이템 삭제, 캐릭터 손에 새로운 무기 스폰 및 소켓에 부착
4. 클라에서 사격 시 서버로 패킷 전송, 서버는 패킷을 브로드캐스트
5. 다른 클라 화면에는 RemoteFire함수가 호출


```

message C_ENTER_TRUCK
{
    uint64 truck_id = 1;
    TruckSeatType seat_type = 2;
}

message S_ENTER_TRUCK
{
    uint64 player_id = 1;
    uint64 truck_id = 2;
    TruckSeatType seat_type = 3;
}

message C_EXIT_TRUCK
{
}

message S_EXIT_TRUCK
{
    uint64 player_id = 1;
    uint64 truck_id = 2;
    TruckSeatType seat_type = 3;
}

message C_TRUCK_MOVE
{
    PosInfo info = 1;
}

message S_TRUCK_MOVE
{
    PosInfo info = 1;
}

```

```

enum TruckSeatType
{
    TRUCK_SEAT_NONE = 0;
    TRUCK_SEAT_DRIVER = 1;
    TRUCK_SEAT_CARGO = 2;
    TRUCK_SEAT_TURRET = 3;
}

```

트럭에 관한 프로토콜이다. 운전석, 트렁크, 총 쏘는 자리로 자리를 나누어서 만들었고, 트럭에 탔을 때, 내렸을 때, 트럭을 운전할 때의 패킷을 만들었다.

Handle_C_ENTER_TRUCK, Handle_C_EXIT_TRUCK, Handle_C_TRUCK_MOVE 패킷이 있는데 패킷 핸들러의 동작원리는 이렇다.

```

auto gameSession = static_pointer_cast<GameSession>(session);

```

PacketSessionRef를 GameSession으로 변환한다.

```

PlayerRef player = gameSession->player.load();
if (player == nullptr)
    return false;

```

세션과 연결된 플레이어 객체가 있는지 스레드 세이프하게 확인한다. 로그인 처리가 안 끝났거나 이미 로그아웃 중인 비정상적인 요청을 걸러낸다.

```

RoomRef room = player->room.load().lock();
if (room == nullptr)
    return false;

```

플레이어가 어떤 방에 들어가 있는 지 확인한다. 룸의 참조를 weak_ptr로 관리하여서 플레이어와 룸이 서로를 shared_ptr로 물고 있는 것을 방지한다.

```
room->DoAsync(&Room::HandleTruckMove, player, Protocol::C_TRUCK_MOVE(pkt));
```

룸의 일감 큐에 DoAsync를 통해 트럭에 관련된 처리를 부탁하고 빠지면 룸을 관리하는 스레드가 순서대로 일을 처리한다.

Room 클래스에서는 HandleEnterTruck, HandleExitTruck, HandleTruckMove 함수를 만들었다.

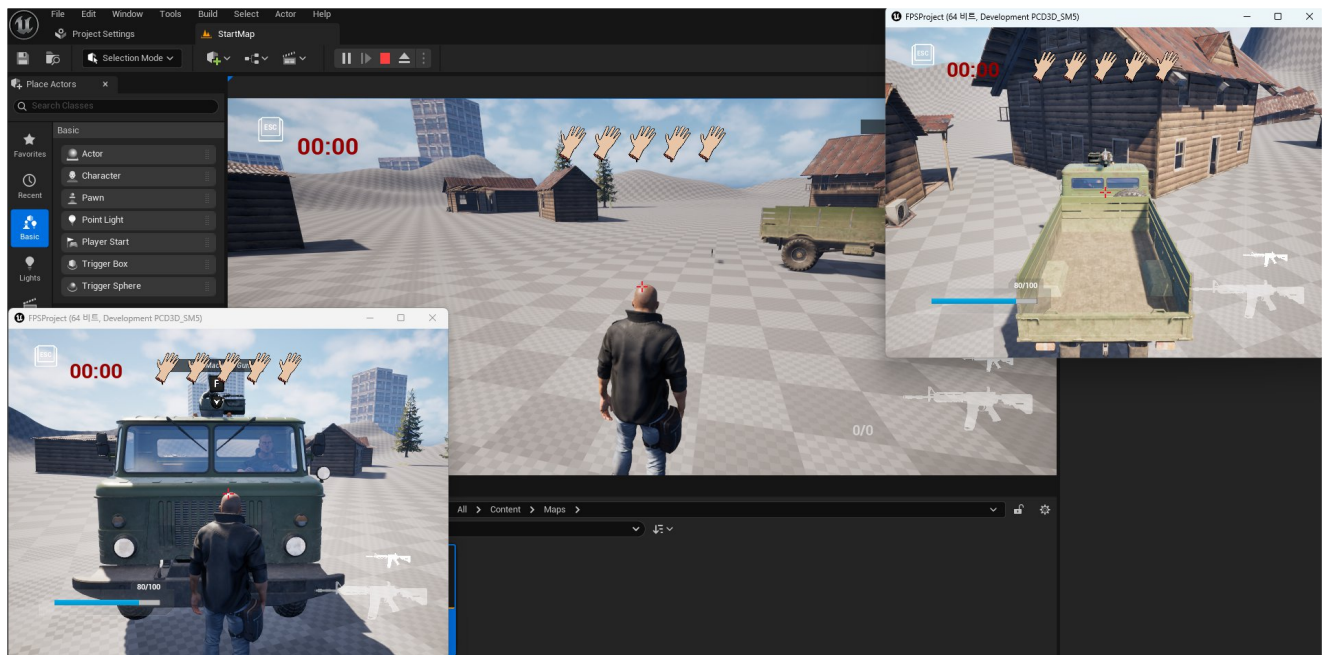
HandleEnterTruck : 클라가 탑승 요청 시 차량의 점유 상태를 검증하고, 이미 다른 플레이어가 있으면 이를 차단한다. 검증이 끝나면 트럭 상태 객체와 플레이어 객체에 서로의 정보를 기록하여 탑승 상태로 만든다.

HandleExitTruck : 플레이어가 내릴 때 트럭과 플레이어쪽의 탑승 상태를 지운다.

HandleTruckMove : 운전자만 트럭을 움직일 수 있게 하고, 트럭의 위치를 플레이어의 위치로 만든다.

클라에서는 S_ENTER_TRUCK, S_EXIT_TRUCK, S_TRUCK_MOVE에서 엔진의 게임 스레드로 작업을 넘겨준다. 함수에서 수신된 패킷 데이터를 new로 복사하고, AsyncTask를 사용하여 처리를 메인 게임 스레드로 넘긴다. 네트워크 스레드에서 액터의 위치를 옮겼을 때 크래시가 나는 것을 방지하기 위함이다.

게임 스레드 내부에서 현재 월드의 gameinstance를 찾아서 실제 처리 함수를 호출하고 복사했던 패킷 메모리를 해제한다.



다음 주 할 일

문 열릴 때 상대방 화면에 브로드캐스트, 트럭에 달린 기관총 브로드캐스트